

ML 위험 탐지 및 검증 대시보드 구성안

1. 페이지 목적

이 페이지는 데이터 자체를 많이 보여주는 곳이 아니라, **보이스피싱 탐지 머신러닝 모델이 어떤 과정을 거쳐 만들어졌고 왜 현재의 2중 모델 구조를 선택했는지**를 한눈에 설명하는 페이지로 구성한다.

핵심 메시지는 다음과 같다.

실제 보이스피싱 통화를 수집·전사한 뒤 정상 금융상담 대조군과 동일한 기준으로 위험신호를 수치화하고, Feature 검증 → Case/Window 학습 데이터 구축 → 다수 알고리즘 비교 → 확률 보정 → Risk Score 및 경고 기준 설정까지 하나의 ML 파이프라인으로 구축하였다.

2. 전체 ML 개발 흐름

대시보드에서는 아래 흐름을 가장 먼저 보여준다.



3. 데이터 출발점

보이스피싱 데이터는 금융감독원 「그놈 목소리」의 실제 음성·영상 자료를 직접 수집하는 방식으로 구축했다.

초기 음성 데이터는 다음 과정을 거쳐 분석 가능한 통화 데이터로 변환했다.

```
음성
↓
Whisper 음성 → 텍스트 전사
↓
Pyannote 화자 분리
↓
범인 / 피해자 역할 판정
↓
발화 Turn 구조화
```

음성 전사에는 `faster-whisper-large-v3-turbo`, 화자 분리에는 `pyannote/speaker-diarization-community-1`을 활용했다. 해당 수집·전사 과정은 프로젝트 작업 기록에도 정리되어 있다.

이후 **정상 금융상담 데이터를 대조군으로 추가하여** 최종 문제를

```
NORMAL = 0
PHISHING = 1
```

의 이진분류 문제로 정의했다.

4. 원문을 바로 머신러닝에 넣지 않은 이유

원문 텍스트만으로는 보이스피싱에서 중요한

- 기관 사칭
- 권위·공권력 이용
- 공포·위협
- 정보 추출
- 민감정보 요구
- 인증정보 요구
- 위험신호의 다양성
- 여러 신호의 동시 출현

등이 숫자로 직접 표현되지 않는다.

따라서 통화 내용에서 **의미 있는 위험신호를 구조화된 Feature로 변환하는 Feature Engineering** 과정을 수행했다.

또한 보이스피싱과 정상 상담에 서로 다른 기준을 적용하면 모델이 데이터 출처 차이를 학습할 가능성이 있으므로, **정상과 피싱 모두 동일한 Feature 추출 기준을 적용했다.**

5. Feature 선정 과정

Feature는 단순히 많이 생성한 뒤 모두 사용하지 않았다.

```
Feature 후보 생성
↓
정상 vs 피싱 동일 기준 추출
↓
통계적 차이 및 예측력 검증
↓
CV 안정성 확인
↓
Semantic QC
↓
최종 Feature 확정
```

특히 정상적인 금융상담에서도

- 금융기관 언급
- 개인정보 확인
- 인증 관련 표현

등이 등장할 수 있기 때문에, 단순 키워드 검출만으로 보이스피싱이라고 판단하지 않도록 Semantic QC를 수행했다.

또한 통화 길이, 텍스트 길이, 출처 자체를 나타내는 변수처럼 **데이터 출처를 간접적으로 구분할 위험이 있는 변수는 모델 Feature에서 제외했다.**

6. 최종 ML Feature

Case Model 기본 Feature — 12개

통화 전체를 하나의 사건으로 판단하기 위한 Feature다.

```
strategy_authority
strategy_fear
imp_public
imp_financial
strategy_info_extraction_sem
sensitive_info_request_sem
auth_info_request_sem
strategy_diversity_sem
action_diversity_sem
signal_family_count_sem
```

```
ix_identityclaim_authority_sem
ix_info_sensitive_sem
```

의미상으로 묶으면 다음과 같다.

- **사칭:** 금융기관 / 공공기관
- **심리 전략:** 권위 / 공포 / 정보추출
- **행동 요구:** 민감정보 / 인증정보
- **복합성:** 전략 다양성 / 행동 다양성 / 위험신호 종류
- **상호작용:** 사칭×권위 / 정보추출×민감정보

Window Model Feature — 12개

실시간 탐지를 위해 현재 통화 구간에서 계산할 수 있는 Feature를 사용한다.

Case Feature와 대부분 동일하지만 전체 통화 기준 `signal_family_count` 대신

```
signal_family_count_delta_sem
```

를 사용한다.

즉 직전 Window와 비교해 위험신호가 증가하고 있는가도 탐지한다.

7. 최종 머신러닝 데이터셋

Case Dataset

```
case_ml_dataset_v1.csv
```

1 Call = 1 Row

- 총 **1,148건**
- 정상 + 보이스피싱 통합
- 최종 Case Feature 12개
- Target: `y_phishing`
- 통화 전체 판별용

Window Dataset

```
window_ml_dataset_v1.csv
```

1 Call → 여러 10-Turn Window

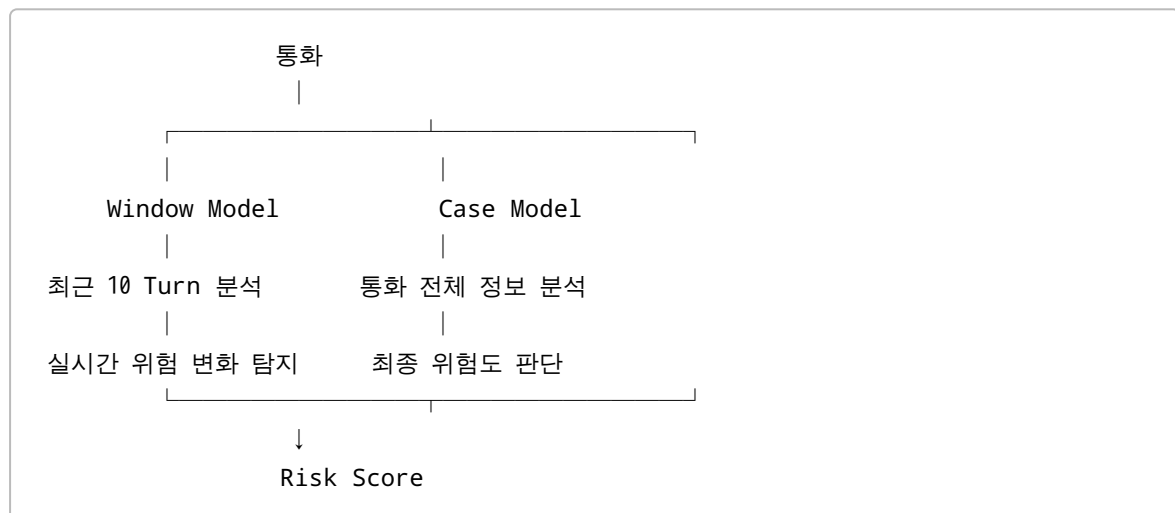
- 총 **7,406 Window**
- 사용 가능한 통화: **968건**
- Window Feature 12개
- **10 Turn Window**
- **Stride = 5 Turn**
- 실시간/조기 탐지용

Window를 임의로 Train/Test에 나누지 않고 먼저 **통화** `source_id` 단위로 **Train / Validation / Test**를 분할한 뒤 해당 통화의 모든 Window가 같은 Split을 상속하도록 했다.

따라서 같은 통화의 일부가 Train에 있고 다른 일부가 Test에 들어가는 데이터 누수를 방지했다.

8. 왜 Case + Window 2중 모델인가?

이 프로젝트의 핵심 구조다.



Window Model

목적:

통화가 끝나기 전에 위험신호를 찾아내는 것

최근 10 Turn을 지속적으로 분석하므로 실시간 경고에 적합하다.

다만 전체 통화 문맥을 보지 못하기 때문에 Case Model보다 판별 난도가 높다.

Case Model

목적:

통화 전체를 이용해 보다 안정적으로 보이스피싱 여부를 판별하는 것

전체 Feature뿐 아니라 Window Model에서 발생한 확률의 흐름도 추가적으로 활용한다.

따라서 두 모델은 경쟁관계가 아니라 역할이 다르다.

Window = 조기경보

Case = 종합판단

9. Case 모델의 Stacking

Case Model에서는 기본 12개 Feature뿐 아니라 Window Model이 통화 과정에서 출력한 위험확률을 요약해 추가 Feature로 사용했다.

STACKED_SAFE 추가 Feature 8개

```
window_prob_mean
window_prob_max
window_prob_p90
window_prob_std
window_prob_first
window_prob_last
window_prob_delta_last_first
window_prob_trend
```

즉 Case Model은 단순히

“어떤 위험신호가 있었는가?”

만 보는 것이 아니라

“통화 진행 중 위험도가 어떤 수준이었고 어떻게 변화했는가?”

까지 함께 본다.

반면

```
window_stack_count
window_stack_available
```

은 통화 길이를 간접적으로 나타내는 Proxy가 될 가능성이 있어 **QC 확인용으로만 남기고 모델 입력에서는 제외했다.**

10. 비교한 머신러닝 알고리즘

동일한 데이터에서 총 5개 알고리즘을 비교했다.

```
Logistic Regression
Decision Tree
Random Forest
XGBoost
LightGBM
```

특정 알고리즘을 처음부터 정답으로 가정하지 않고 Validation 성능을 기준으로 비교했다.

11. 모델 선정 기준

보이스피싱처럼 Positive 탐지가 중요한 문제에서는 Accuracy만 사용하면 부적절할 수 있다.

따라서 **Validation PR-AUC를 1차 모델 선정 기준**으로 사용했다.

추가적으로 다음 지표를 함께 확인했다.

- PR-AUC
- ROC-AUC
- Precision
- Recall
- F1
- Brier Score
- Confusion Matrix

왜 PR-AUC인가?

보이스피싱 탐지에서는

```
실제 피싱을 얼마나 놓치지 않는가
+
피싱이라고 판단한 것 중 실제 피싱 비율이 얼마나 높은가
```

가 중요하다.

따라서 Precision과 Recall의 관계를 전체 Threshold에서 평가하는 **PR-AUC를 주요 비교 지표로 선정**했다.

12. Window 모델 비교 결과

Validation PR-AUC 기준:

알고리즘	Validation PR-AUC
RandomForest	0.8176
LightGBM	0.8136
XGBoost	0.8133
Logistic Regression	0.8131
Decision Tree	0.8026

최종 선택:

Window Model = RandomForest

성능:

- Validation PR-AUC: **0.8176**
- Test PR-AUC: **0.7816**

Window 모델은 전체 통화를 보는 모델이 아니라 **현재까지 관찰된 제한적인 구간만 가지고 판단하기 때문에** Case Model과 PR-AUC 수치를 직접 우열 비교해서는 안 된다.

13. Case 모델 비교 결과

Case 모델에서는 두 가지 Feature Set을 비교했다.

BASE

Case 기본 Feature 12개

STACKED_SAFE

Case 기본 Feature 12개
+
Window 확률 요약 Feature 8개

총 20개 Feature.

상위 결과:

Feature Set / Model	Validation PR-AUC
STACKED_SAFE / RandomForest	0.9561
STACKED_SAFE / XGBoost	0.9408
STACKED_SAFE / Logistic Regression	0.9390

Feature Set / Model	Validation PR-AUC
STACKED_SAFE / LightGBM	0.9276
BASE / XGBoost	0.9227
BASE / RandomForest	0.9214

따라서 최종 선택은:

Case = STACKED_SAFE + RandomForest

14. 최종 모델 성능

Window 최적 모델

RandomForest

- Validation PR-AUC: **0.8176**
- Test PR-AUC: **0.7816**

Case 최적 모델

RandomForest + STACKED_SAFE

- Validation PR-AUC: **0.9561**
- Test PR-AUC: **0.9582**

Case Test 추가 성능

지표	결과
PR-AUC	0.9582
ROC-AUC	0.9248
Precision	0.8559
Recall	0.8783
F1	0.8670

Validation과 Test PR-AUC가 각각 **0.9561 / 0.9582**로 큰 차이를 보이지 않았다는 점도 내부 데이터 기준에서는 안정적인 결과다.

15. 모델 확률 → Risk Score

Feature마다 사람이 임의로 점수를 부여하지 않는다.

즉 다음과 같은 방식은 사용하지 않는다.

```
사칭 +20점  
긴급성 +15점  
공포 +10점  
...
```

실제 Risk Score는 머신러닝 모델이 학습한 확률을 기반으로 한다.

최종 공식은 매우 단순하다.

```
Raw Probability  
↓  
Calibration  
↓  
Calibrated Probability  
↓  
Risk Score = Calibrated Probability × 100
```

수식

```
Risk Score = Pcalibrated(PHISHING) × 100
```

예를 들어 보정된 피싱 확률이

```
0.73
```

이라면

```
Risk Score = 73점
```

이다.

16. Calibration

머신러닝의 `predict_proba()` 값이 실제 확률과 완전히 일치한다고 보장할 수 없기 때문에 Calibration을 별도로 수행했다.

비교 방법:

- Identity — 보정 없음
- Platt Scaling
- Isotonic Regression

Validation OOF 확률을 이용해

- Brier Score
- Log Loss
- Calibration Error

등을 비교했다.

최종:

```
Window → Platt Scaling
Case   → Identity
```

즉 Window 모델은 확률 보정을 적용했고, Case 모델은 보정하지 않은 원래 확률을 사용하는 것이 내부 검증에서 가장 적합했다.

17. Alert Threshold

고정적으로 0.5 를 사용하지 않았다.

Validation 데이터에서

- Recall
- FPR
- Precision
- F1
- F2

의 Trade-off를 비교해 경고 Threshold를 탐색했다.

현재 내부 설정:

```
Window Alert Risk Score = 10
Case Alert Risk Score   = 39
```

즉:

```
Window calibrated probability ≥ 0.10
→ 경고
```

Case calibrated probability ≥ 0.39

→ 경고

다만 이 Threshold는 현재 내부 데이터 기준의 프로토타입 운영값이다.

특히 Recall을 매우 높게 가져가는 과정에서 False Positive가 많이 발생하므로, 실제 서비스에 바로 확정값으로 사용하면 안 된다.

외부 데이터 확보 후 실제 오탐 비용과 미탐 비용을 반영하여 재Calibration 및 Threshold 재설정이 필요하다.

18. 최종 배포 구조

최종적으로 서비스에서는 별도의 여러 모델 파일을 직접 조합하지 않고 다음 구성요소를 하나의 Pipeline으로 묶는다.

```
Window RandomForest
+
Case RandomForest
+
Window → Case Stacking 정보
+
Feature 목록
+
Calibration
+
Alert Threshold
+
Risk Score 공식
```

최종 모델 파일:

```
final_voice_phishing_risk_pipeline.pkl
```

실제 서비스에서는:

```
최근 10 Turn
↓
Feature 추출
↓
Window RandomForest
↓
Window Risk 갱신
↓
Case / Stacking
```

↓
Calibration
↓
Risk Score
↓
Threshold 초과 시 경고

형태로 사용할 수 있다.

19. 대시보드에서 반드시 보여줄 핵심만 선별

이 페이지는 내용을 너무 많이 넣지 않는다.

A. 상단 — ML 개발 Pipeline

원천 통화
→ 전사/화자분리
→ EDA
→ Feature Engineering
→ Semantic QC
→ ML Dataset
→ 모델 비교
→ Calibration
→ Risk Score

가로 Flow로 표시한다.

B. 최종 데이터셋 카드

Case Dataset

1,148 Calls

1 Call = 1 Row

통화 전체 판별

Window Dataset

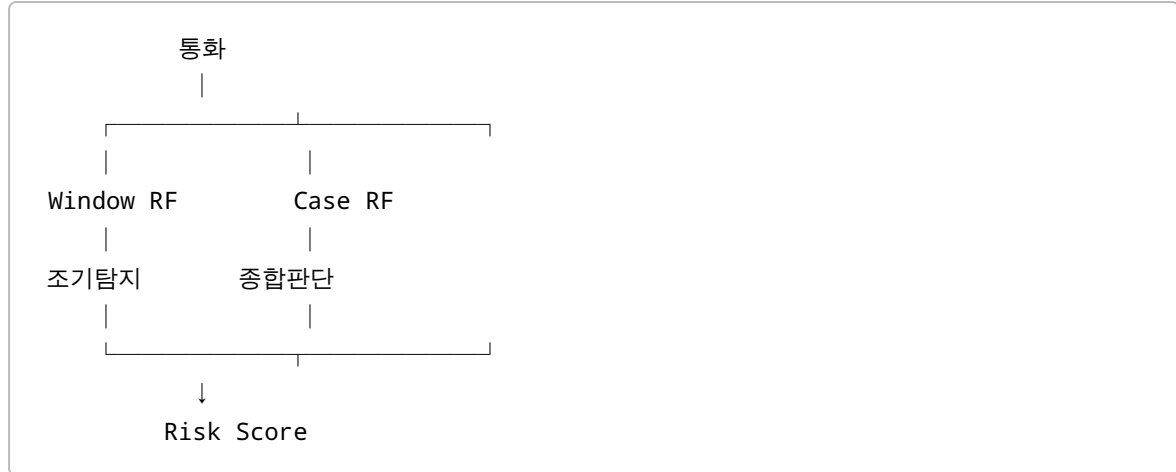
7,406 Windows

10 Turn / Stride 5

실시간 조기탐지

C. 2중 모델 Architecture

가장 직관적으로 보여준다.



아래 설명:

Case 하나만 사용하면 통화 종료 전 경고가 어렵고, Window 하나만 사용하면 전체 문맥이 부족하다. 따라서 조기탐지와 종합판단을 분리한 2중 구조를 사용한다.

D. 모델 비교 그래프

Window

5개 모델의 Validation PR-AUC Horizontal Bar.

RandomForest를 강조한다.

Case

BASE / STACKED_SAFE 와 5개 알고리즘 비교 결과를 표시한다.

너무 많은 숫자를 표시하지 말고 상위 모델과 RandomForest를 중심으로 보여준다.

E. 최종 성능 카드

WINDOW

RandomForest

- VAL PR-AUC **0.8176**
- TEST PR-AUC **0.7816**

CASE

RandomForest / STACKED_SAFE

- VAL PR-AUC **0.9561**
- TEST PR-AUC **0.9582**

추가:

ROC-AUC	0.9248
Precision	0.8559
Recall	0.8783

F. Risk Score Flow

Model Probability
↓
Calibration
↓
Risk Score ×100
↓
Alert Threshold
↓
실시간 경고

작게 수식 표시:

$$\text{Risk Score} = \text{Calibrated } P(\text{Phishing}) \times 100$$

20. 이 페이지에서 빼는 내용

대시보드를 논문처럼 만들 필요는 없다.

다음은 화면에서 제외하거나 Tooltip 수준으로만 사용한다.

- 모든 12/20개 Feature의 긴 테이블
- Feature별 모든 p-value
- 모든 CV Fold 결과
- Confusion Matrix 여러 개
- 모든 Brier Score
- 전체 Threshold Curve 숫자
- 전체 Semantic QC 결과

- 모든 Feature Importance
- 모든 중간 CSV
- 전처리 코드
- 학습 코드
- 모든 Hyperparameter

핵심은 “어떻게 만들었는가 → 왜 이 구조인가 → 무엇을 비교했는가 → 무엇이 최종 선택됐는가 → 서비스에서는 어떻게 점수화되는가”다.

21. 해석상 반드시 표시할 한계

성능을 과장해서 표현하면 안 된다.

대시보드 하단에 작은 안내문으로 다음 내용을 넣는다.

현재 성능은 내부 데이터 기준 검증 결과이며 완전히 독립된 외부 Test 결과는 아니다. Feature 선정 과정에서 동일 출처 데이터가 사용되었고 Source Confounding이 완전히 해결되었다고 볼 수 없으므로, 실제 서비스 적용 전 별도 외부 데이터 검증과 Calibration이 필요하다.

또한 현재 Threshold 역시 실제 운영 확정값이 아니라 내부 Prototype 기준임을 표시한다.

22. 코덱스 구현 요청사항

참고 Drive:

https://drive.google.com/drive/u/1/folders/10osv7mAsgrEJ1g_hoxpiZOCjcsDkWBD_

참고 Notion:

<https://app.notion.com/p/hambrojun/3bac753ff28a8052992dd542fb0c1b57>

특히 다음 자료를 우선 참고한다.

- 04 Feature 찾기
- 05 ML Dataset
- 06 머신러닝 모델링(다수 모델 비교)
- 07 Calibration_RiskScore_Threshold

구현 원칙

1. 기존 대시보드 Sidebar 및 전체 디자인 언어를 유지한다.
2. 페이지 제목은 「ML 위험 탐지 및 검증」으로 유지한다.
3. 데이터 그래프보다 머신러닝 제작 과정이 주인공이 되게 구성한다.
4. 처음부터 끝까지 하나의 스토리로 읽히게 한다.
5. 수치 및 모델 결과는 가능하면 결과 CSV/JSON에서 불러오고 불필요한 하드코딩을 피한다.

6. RandomForest만 보여주지 말고 **5개 알고리즘을 실제 비교한 뒤 선택했다는** 과정을 보여준다.
 7. Case와 Window PR-AUC를 직접 우열 비교하는 표현은 하지 않는다.
 8. `STACKED_SAFE`가 무엇인지 반드시 간단히 설명한다.
 9. Risk Score가 Feature별 수동 가중치 합산이 아니라 **Calibrated Probability × 100**이라는 점을 명확히 한다.
 10. 현재 Threshold를 절대적인 운영 기준처럼 표현하지 않는다.
 11. 내부 Test의 한계와 외부 재검증 필요성을 작은 안내문으로 표시한다.
-

최종 화면에서 전달해야 할 한 문장

실제 통화에서 의미 있는 위험신호를 추출·검증해 Case와 Window 두 종류의 학습 데이터를 구축하고, 5개 알고리즘 비교 결과 RandomForest를 최종 선택했다. Window 모델은 실시간 조기탐지, Case 모델은 전체 문맥과 Window 위험 흐름을 결합한 종합판단을 담당하며, 예측확률은 Calibration 후 0~100 Risk Score로 변환해 서비스 경고에 활용한다.